Segment Trees

1. Zooming in on Line Segments
   - data structures for windowing queries
   - the locus approach to develop data structures
   - a binary search tree for intervals
   - running an example of CGAL
2. Segment Trees
   - definition and storage cost
   - query a segment tree
   - constructing a segment tree
   - associated data structures

MCS 481 Lecture 32
Computational Geometry
Jan Verschelde, 6 November 2026

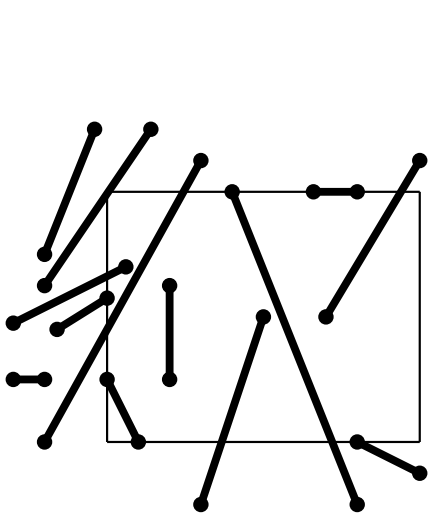
Segment Trees

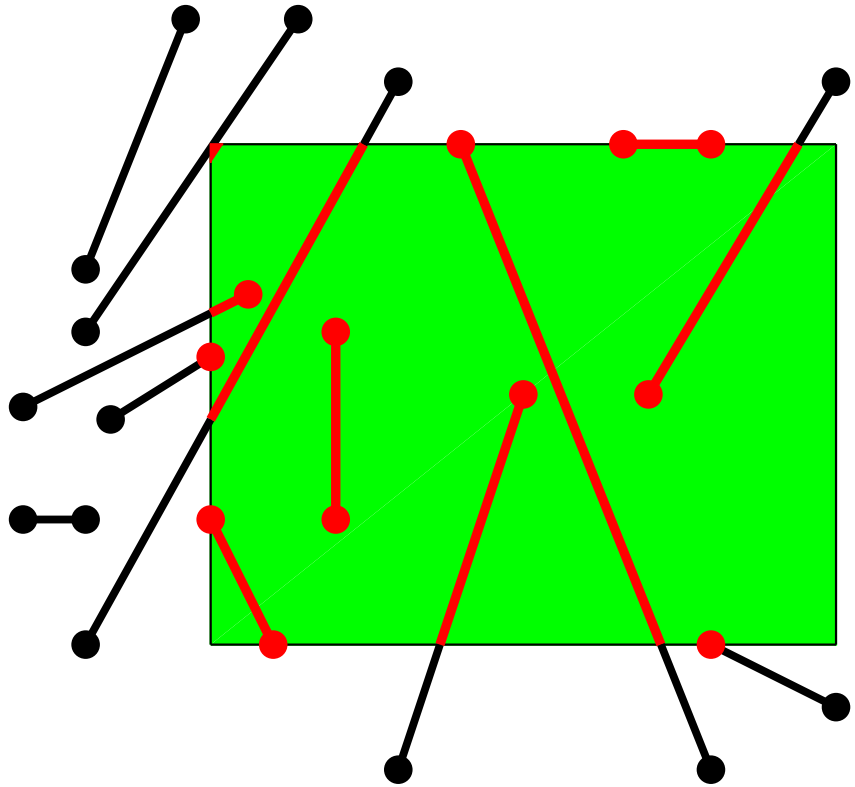
1. Zooming in on Line Segments
   - data structures for windowing queries
   - the locus approach to develop data structures
   - a binary search tree for intervals
   - running an example of CGAL

2. Segment Trees
   - definition and storage cost
   - query a segment tree
   - constructing a segment tree
   - associated data structures

windowing queries

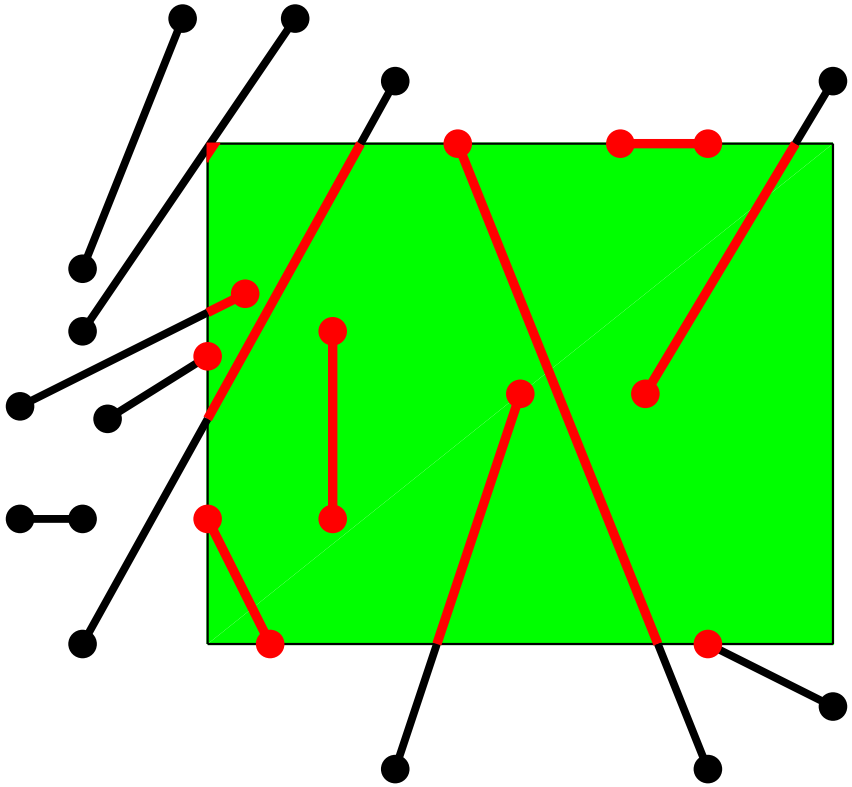
Given a map, we zoom in on a window $\Rightarrow$ windowing query.





We assume the line segments do not intersect each other.

types of segments

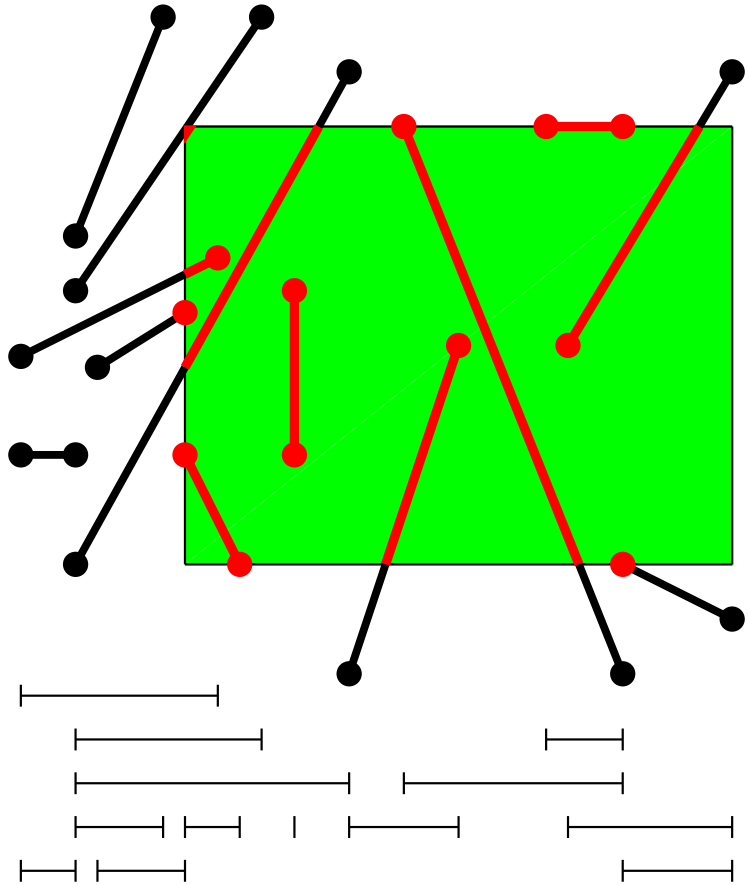


Two types of segments:

1. an end point lies inside the query window (use range tree),
2. segments intersect the boundary of the window.

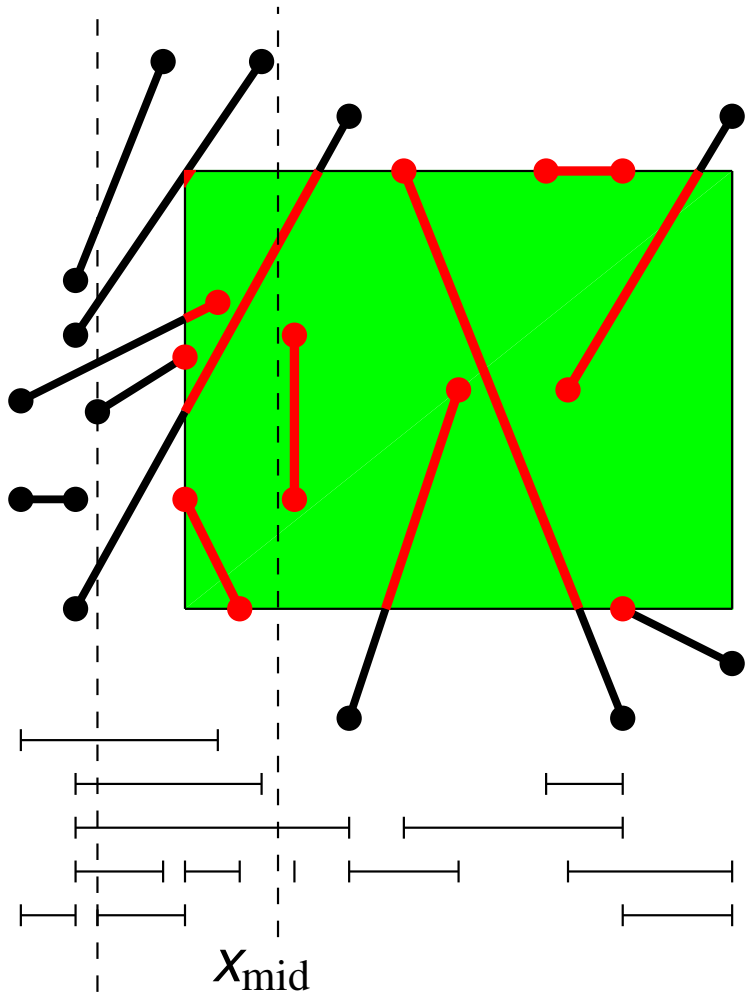
a vertical query segment

Consider a vertical query segment $q_x \times [q_y : q'_y]$.



The x-coordinates of the end points of the segments define intervals.

an interval tree



Does intersection with $x = x_{\text{mid}}$ imply that in window?

Segment Trees

1. Zooming in on Line Segments
   - data structures for windowing queries
   - the locus approach to develop data structures
   - a binary search tree for intervals
   - running an example of CGAL

2. Segment Trees
   - definition and storage cost
   - query a segment tree
   - constructing a segment tree
   - associated data structures

developing data structures with the locus approach

Our query window $W = [q_x : q'_x] \times [q_y : q'_y]$ has four parameters.

Observe: for nearby choices of W, we often get the same answer.

The *locus approach* suggests

- to partition the parameter space into regions,
- such that queries in the same region have the same answer.

We apply the locus approach to develop an alternative to interval trees.

Segment Trees

1. Zooming in on Line Segments
   - data structures for windowing queries
   - the locus approach to develop data structures
   - a binary search tree for intervals
   - running an example of CGAL

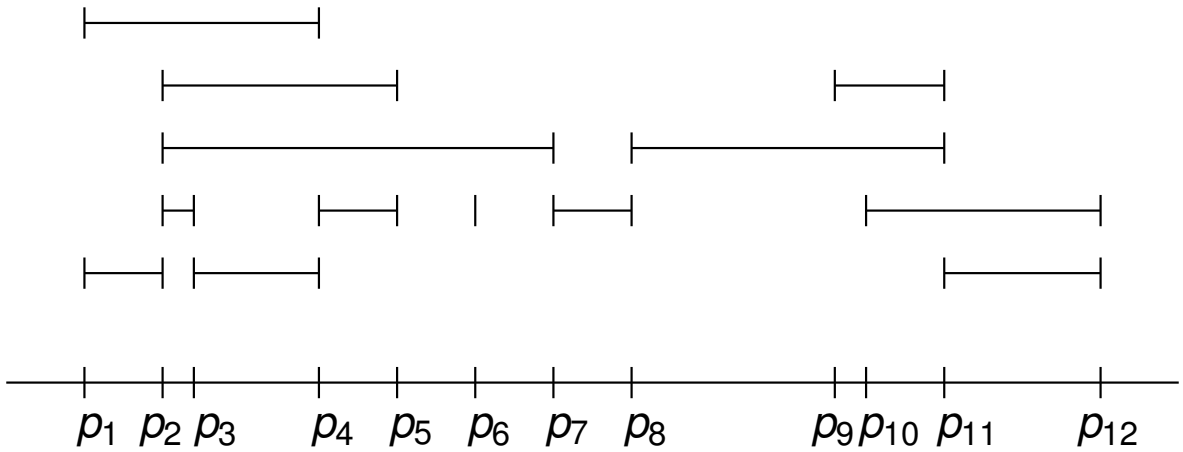
2. Segment Trees
   - definition and storage cost
   - query a segment tree
   - constructing a segment tree
   - associated data structures

organizing a set of intervals

Given is a set I of n intervals:

$$I = \{[x_1 : x_1'], [x_2 : x_2'], \ldots, [x_n : x_n']\}.$$

The n intervals define m distinct points:



The m points are sorted $p_1 < p_2 < \cdots < p_m$ and define *the elementary intervals* (observe: ( ) is open [ ] is closed):

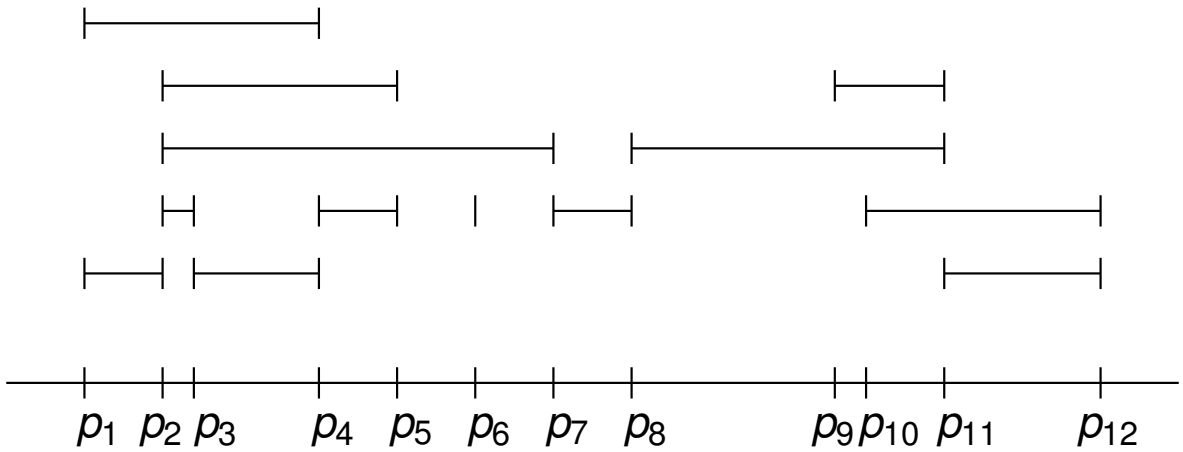
$$(-\infty : p_1), [p_1 : p_1], (p_1 : p_2), \ldots, [p_m : p_m], (p_m : +\infty).$$

query the elementary intervals

Given a query point q_x and the elementary intervals

$$(-\infty : p_1), [p_1 : p_1], (p_1 : p_2), \ldots, [p_m : p_m], (p_m : +\infty),$$

determine the elementary interval that contains q_x.

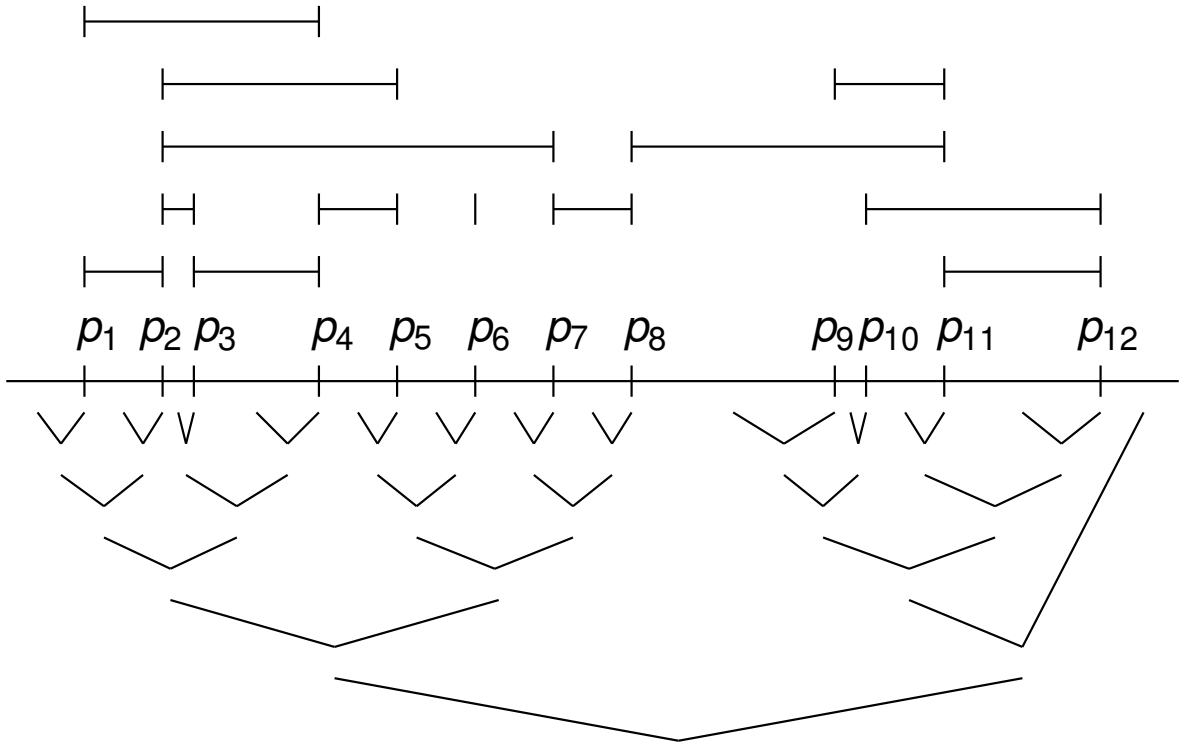


How to organize a binary search tree for such a query?

a binary search tree

The leaves of the search tree are the elementary intervals

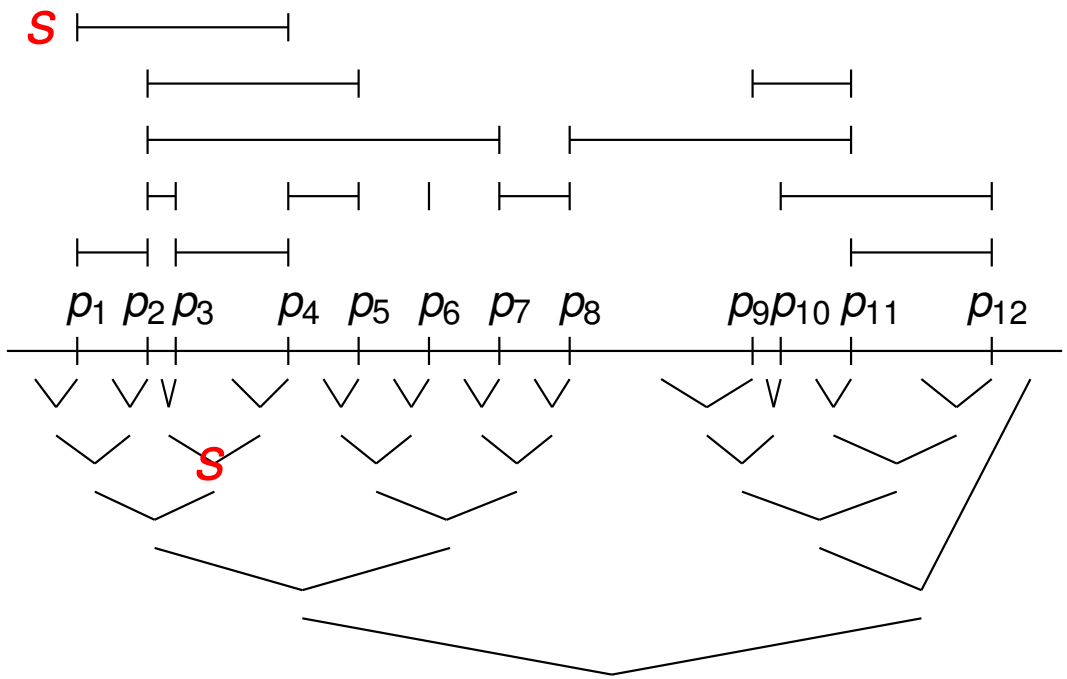
$$(-\infty : p_1), [p_1 : p_1], (p_1 : p_2), \ldots, [p_m : p_m], (p_m : +\infty).$$



The intervals that contain an elementary interval are at its leaf.

principle of a segment tree

If an interval spans many elementary intervals, then the interval will be repeated many times and the storage cost will be high.



Principle of *a segment tree*: store the interval in a node

as close to the root as possible .

Exercise 1: complete the picture above, placing all segments at the nodes in the tree, following the principle of a segment tree.

all intervals overlap

Exercise 2: Consider $I = \{[0:5], [1:6], [2:7], [3:8], [4:9]\}$.

1. Define the binary tree to store all intervals.
2. In this example, all intervals overlap.
   Can the storage grow quadratically in the number of intervals?
   Justify your answer, referring to your tree for this example.

Segment Trees

1. Zooming in on Line Segments
   - data structures for windowing queries
   - the locus approach to develop data structures
   - a binary search tree for intervals
   - running an example of CGAL

2. Segment Trees
   - definition and storage cost
   - query a segment tree
   - constructing a segment tree
   - associated data structures

running an example of CGAL

On a Window Subsystem for Linux (WSL), running Ubuntu:

1. Run `sudo apt-get install` followed by `libgmp-dev`, `libmpfr-dev`, `libcgal-dev`.
2. Download `CGAL-5.6-examples.zip`.
3. Compile `segment_tree_map_2.cpp` from the `RangeSegmentTrees` folder.

This example constructs a segment tree and performs a window query.

Exercise 3: Construct trees for sufficiently large point sets.
Report the observed times for the construction of the data structure.
Do you observe the $O(n \log(n))$ as n grows?

Segment Trees

1. Zooming in on Line Segments
   - data structures for windowing queries
   - the locus approach to develop data structures
   - a binary search tree for intervals
   - running an example of CGAL

2. Segment Trees
   - definition and storage cost
   - query a segment tree
   - constructing a segment tree
   - associated data structures

definition of a segment tree

Given is a set I of n intervals.
The m end points of the intervals are sorted and
define *the elementary intervals*.

- The leaves of a balanced binary tree T store the elementary intervals, in the sorted order of the end points:
  - the leftmost leaf of T stores the leftmost interval,
  - the rightmost leaf of T stores the rightmost interval.

- Each internal node of T is the union of the elementary intervals at the leaves under the node.
  The interval at a node is the union of the intervals of its children.

More than one interval is stored at a node ...

definition of a segment tree — the canonical subset

. . . what is stored at the nodes:

- Each node v stores an interval and a set $I(v)$ of intervals.

  This *canonical subset* $I(v)$ at the node v contains all $[x : x'] \in I$:
  - the interval of the node v is contained in $[x : x']$, and
  - the interval of the parent of v is not contained in $[x : x']$.

Exercise 4: verify that the above definition of the canonical subset agrees with the principle of a segment tree, refer to Exercise 1.

storage cost

Lemma (storage cost of a segment tree)

A segment tree on a set of n intervals occupies $O(n \log(n))$ storage.

- The leaves of the tree are the elementary intervals. How many?

  Each end point is stored separately, for n given intervals, we have
  - at most $2n$ intervals of type $[p : p']$, for end points p,
  - at most $2n - 1$ intervals in between $(p_i : p_{i+1})$, and
  - the intervals $(-\infty : p_1)$ and $(p_m : +\infty)$.

  If all endpoints are distinct from each other, then $m = 2n$.

  In total, we have at most $4n + 1$ leaves in the tree.

- The tree is a balanced binary search tree, of depth $O(\log(n))$.

- But how many times does each given interval occur in the tree?

on the canonical subset

From the definition of the segment tree:

- Each node v stores an interval and a set $I(v)$ of intervals.
  This *canonical subset* $I(v)$ at the node v contains all $[x : x'] \in I$:
  - the interval of the node v is contained in $[x : x']$, and
  - the interval of the parent of v *is not contained* in $[x : x']$.

The second item ensures every given interval is stored at most twice.

Assume otherwise, $[x : x']$ is stored at nodes v_1, v_2, and v_3,
all at the same depth.

1. $[x : x']$ spans the whole interval at v_2,
   from the left end point of the interval stored at v_1,
   from the right end point of the interval stored at v_3.
2. Because v_2 lies between v_1 and v_3,
   the interval at the parent of v_2 is contained in $[x : x']$.

Therefore, $[x : x']$ is not stored at v_2, contradicting the assumption.

Q.E.D.

Segment Trees

1. Zooming in on Line Segments
   - data structures for windowing queries
   - the locus approach to develop data structures
   - a binary search tree for intervals
   - running an example of CGAL

2. Segment Trees
   - definition and storage cost
   - query a segment tree
   - constructing a segment tree
   - associated data structures

query a segment tree

Algorithm QUERYSEGMENTTREE(v, q_x)

Input: v is the root of a segment tree T,
q_x is a query point.
Output: all intervals in T containing q_x.

1. report all intervals in $I(v)$
2. if v is not a leaf then
3. if q_x is in the interval of LEFTCHILD(v) then
4. QUERYSEGMENTTREE(LEFTCHILD(v, q_x))
else
5. QUERYSEGMENTTREE(RIGHTCHILD(v, q_x))

an output sensitive algorithm

Lemma (the cost of a query of a segment tree)

Algorithm QUERYSEGMENTTREE *reports k intervals in a segment tree on n intervals in time $O(\log(n) + k)$.*

- The depth of the balanced search tree is $O(\log(n))$.
- At node v, we spend $O(1 + k_v)$ time, where k_v is the number of reported intervals at node v.

The sum of all k_v's equals k.

Segment Trees

1. Zooming in on Line Segments
   - data structures for windowing queries
   - the locus approach to develop data structures
   - a binary search tree for intervals
   - running an example of CGAL

2. Segment Trees
   - definition and storage cost
   - query a segment tree
   - constructing a segment tree
   - associated data structures

constructing a segment tree

Algorithm INSERTSEGMENTTREE(v, $[x : x']$)

Input: v is the root of a segment tree T,
$[x : x']$ is the interval to be inserted in T.
Output: v where $[x : x']$ is inserted.

1. If the interval at v is contained in $[x : x']$ then
2. store $[x : x']$ at v

else

3. if the interval at LEFTCHILD(v) $\cap$ $[x : x'] \neq \emptyset$ then
4. INSERTSEGMENTTREE(LEFTCHILD(v), $[x : x']$)
5. if the interval at RIGHTCHILD(v) $\cap$ $[x : x'] \neq \emptyset$ then
6. INSERTSEGMENTTREE(RIGHTCHILD(v), $[x : x']$)

construction cost

A balanced binary tree is constructed on the elementary intervals, Algorithm INSERTSEGMENTTREE inserts the intervals.

Lemma (cost to insert one interval in a segment tree)

Inserting one interval in a segment tree on n intervals runs in $O(\log(n))$ time.

Summarizing:

Theorem (performance of a segment tree)

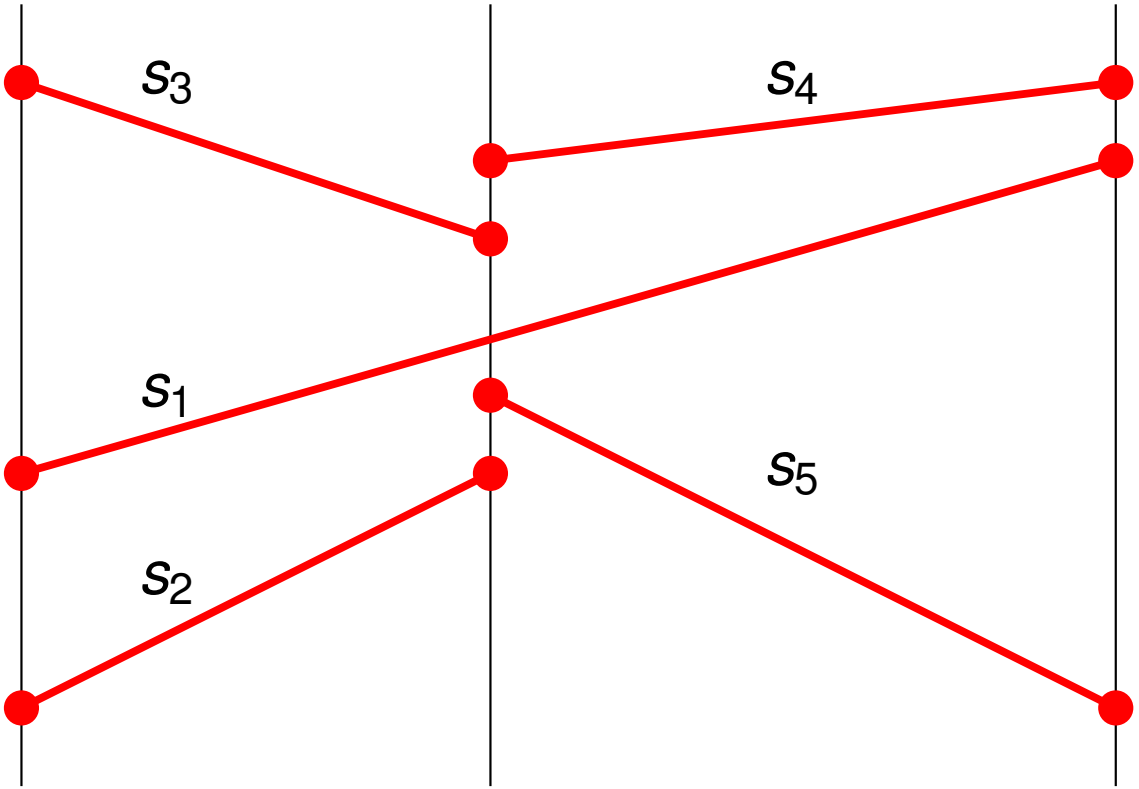
For n intervals a segment tree takes $O(n \log(n))$ storage and $O(n \log(n))$ time to construct. The query time is $O(\log(n) + k)$, where k is the number of reported intervals.

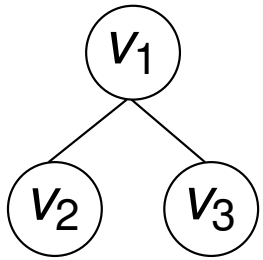
Segment Trees

1. Zooming in on Line Segments
   - data structures for windowing queries
   - the locus approach to develop data structures
   - a binary search tree for intervals
   - running an example of CGAL

2. Segment Trees
   - definition and storage cost
   - query a segment tree
   - constructing a segment tree
   - associated data structures

back to the windowing problem





$$S(v_1) = \{s_1\}$$
$$S(v_2) = \{s_2, s_3\}$$
$$S(v_3) = \{s_4, s_5\}$$

- A segment spans a slab. Segments are stored in a segment tree.
- Order the segments in a slab vertically.

associated data structure

At each node of the segment tree, we store a binary search tree, based on the vertical order of the segments within the slab of the segment.

A vertical query segment is $q_x \times [q_y : q'_y]$.

Theorem (vertical query segments)

Let S be a set of n disjoint line segments in the plane.
Segments intersecting with a vertical query segment $q_x \times [q_y : q'_y]$

- *can be reported in $O(\log^2(n) + k)$ time, where k is the number of reported segments,*
- *with a data structure using $O(n \log(n))$ storage,*
- *built in time $O(n \log(n))$.*

summary and exercises

Application of the locus approach leads to segment trees.

In $O(n \log(n))$ time and space, windowing with vertical query segments can be done in time $O(\log^2(n) + k)$, where k is the size of the output.

We covered section 10.3 in the textbook.

Consider the following activities, listed below.

1. Write the solutions to exercises 1, 2, 3, and 4.
2. Read the CGAL documentation on the Segment Trees.
3. Consider the exercises 5, 7, 8 in the textbook.